

AllianceMine Build and Release — Complete Documentation

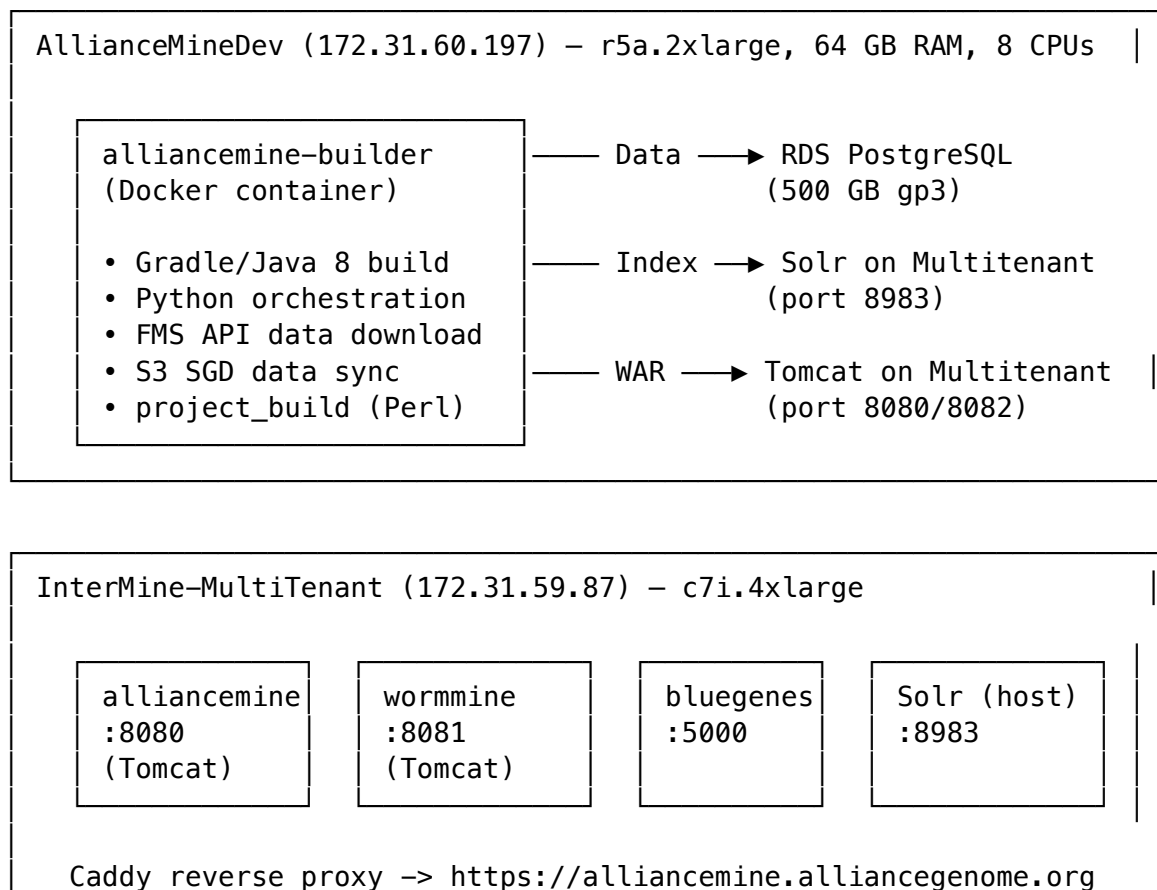
Table of Contents

1. [System Overview](#)
 2. [Infrastructure](#)
 3. [Data Sources](#)
 4. [Build Pipeline](#)
 5. [Data Extraction](#)
 6. [Database Management](#)
 7. [Solr Search Indexes](#)
 8. [WAR Deployment](#)
 9. [Release Cutover](#)
 10. [Monitoring](#)
 11. [Troubleshooting](#)
-

1. System Overview

AllianceMine is an InterMine data warehouse that integrates genomic data from all Model Organism Databases (MODs) in the Alliance of Genome Resources. The build system creates the database, downloads and integrates data, creates search indexes, and deploys the web application.

Architecture



```
AWS RDS PostgreSQL (intermine-postgres, 500 GB gp3)
```

```
alliancemine_9_0_0      (production, ~57 GB)
alliancemine_userprofile (persistent across releases, ~18 MB)
alliancemine_items      (staging, rebuilt each build)
wormmine_final          (~38 GB)
```

SSH Access

Both machines are accessed via the same key:

```
# AllianceMineDev (build machine)
ssh -i ~/.ssh/AGR-ssl3.pem ec2-user@172.31.60.197

# InterMine-MultiTenant (production)
ssh -i ~/.ssh/AGR-ssl3.pem ec2-user@172.31.59.87
```

WARNING: Never stop containers on the Multitenant — it serves production traffic.

2. Infrastructure

AllianceMineDev (Build Machine)

Property	Value
IP	172.31.60.197
Instance	r5a.2xlarge
RAM	64 GB
CPUs	8
Disk	750 GB (217 GB free as of last build)
Docker	25.x with compose plugin
Purpose	Running builds, testing

InterMine-MultiTenant (Production)

Property	Value
IP	172.31.59.87
Instance	c7i.4xlarge
Containers	alliancemine (:8080), wormmine (:8081), bluegenes (:5000)
Solr	Running on host, port 8983
Proxy	Caddy -> HTTPS
Purpose	Serves production at alliancemine.alliancegenome.org

RDS PostgreSQL

Property	Value
Endpoint	intermine-postgres.cmnnhlso7wdi.us-east-1.rds.amazonaws.com
Storage Engine	500 GB gp3
Encoding	PostgreSQL 15
Max connections	UTF-8
	250

External: SGD Database (Read-Only)

Property	Value
Host	www-rds-primary.yeastgenome.org
Database	sgd
Port	5432
Purpose	SGD gene data, complexes, complementation
Credentials	In .env on AllianceMineDev only

3. Data Sources

Alliance FMS (File Management System)

49 files downloaded via the FMS snapshot API. The `extract_data.py` script resolves S3 URLs from the snapshot and downloads/decompresses them.

Category	Files	Format	Target Dir
BGI (Basic Gene Info)	9 (per MOD)	JSON	/root/data/genes/
GFF (Genomic Features)	9 (per MOD)	GFF3	/root/data/fms/
GAF (GO Annotations)	8 (per MOD)	GAF	/root/data/fms/
FASTA (Genome Seq)	7 (per assembly)	FASTA	/root/data/fms/
Ontologies	12 (DO, GO, ECO, etc.)	OBO	/root/data/fms/
Disease	1 (COMBINED)	TSV	/root/data/fms/
Orthology	1 (COMBINED)	TSV	/root/data/fms/
Expression	1 (COMBINED)	TSV	/root/data/fms/
Variants/Alleles	1 (COMBINED)	TSV	/root/data/fms/

S3: SGD-Specific Data

Downloaded from `s3://agr-db-backups/alliancemine/intermine/` via `aws s3 cp --recursive`.

Directory	Contents
ontology/ gff/	psi-mi.obo, goslim_yeast.obo sgd_transcriptome_v1.gff

Directory	Contents
gff-utr/	sgd_gff_utrs.tsv
db-utr/	db_utrs_from_ym.tsv
protein-properties/	wallace2015_heat_aggregation_annotations_Kyla.txt
protein-ntermini/	nTermStarts.txt
protein-modifications/	allProteinModifications.txt
yeast_orthologs/	fungidb, CGOB, C.glabrata, pombe, homolog_genes
idresolver/	entrez, wormids

Important: Do NOT download `protein_properties.tab` from `downloads.yeastgenome.org` — it has 40 columns and crashes the protein-properties converter which expects 7.

SGD PostgreSQL Database

Three InterMine sources read directly from SGD's database: - `sgd` — main gene/feature data (SgdConverter) - `sgd-complementation-db` — complementation data - `sgd-complexes` — protein complex data

Alliance API (replaces FMS feeds)

Nine TSVs produced by Python fetchers in `alliancemine-bio-sources/scripts/`, calling `https://www.alliancegenome.org/api/` directly. Land in `/root/data/api/`.

TSV	Source endpoint	Bio-source module
alliance-genes.tsv	/gene/{id}	alliance-genes (enriched)
genetic-interactions.tsv	/gene/{id}/genetic-interactions	alliance-genetic-interactions
molecular-interactions.tsv	/gene/{id}/molecular-interactions	alliance-molecular-interactions
paralogs.tsv	/gene/{id}/paralogs	alliance-paralogs
phenotypes.tsv	/gene/{id}/phenotypes	alliance-phenotypes
disease-models.tsv	/gene/{id}/models (non-yeast MODs)	alliance-disease-models
allele-detail.tsv	/allele/{id}	alliance-allele-detail
orthologs.tsv	/gene/{id}/orthologs	alliance-ortholog-detail
disease-annotations-detail.tsv	/disease/{id}/genes	alliance-disease-detail

The fetchers cache responses in a SQLite store at `/root/data/api-cache/` (via `ALLIANCE_FETCH_CACHE`). Cold cache ~20 min for yeast-only, hours for multi-MOD; warm reruns ~2 min. The cache survives container recreation since it lives under the `./data:/root/data` bind-mount.

See `docs/API_FETCHER_INTEGRATION.md` for orchestration, failure modes, and verification checks.

4. Build Pipeline

Configuration

All configuration is in `docker/alliancemine/.env` on AllianceMineDev:

```
# RDS
RDS_HOST=intermine-postgres.cmnhlso7wdi.us-east-1.rds.amazonaws.com
RDS_PORT=5432
RDS_USER=postgres
RDS_PASSWORD=<secret>

# Build
ALLIANCE_RELEASE=9.0.0
BUILD_TYPE=test          # test = auto RC numbering, production = release naming
FMS_RELEASE_TYPE=current # or "next"

# SGD Database
SGD_DB_HOST=www-rds-primary.yeastgenome.org
SGD_DB_NAME=sgd
SGD_DB_USER=<user>
SGD_DB_PASSWORD=<secret>
SGD_DB_PORT=5432

# Solr (versioned cores on multitenant)
SOLR_INDEX_URL=http://172.31.59.87:8983/solr/alliancemine-search-9.0.0
SOLR_AUTOCOMPLETE_URL=http://172.31.59.87:8983/solr/alliancemine-autocomplete-9.0.0

# Webapp
SUPERUSER_ACCOUNT=superuser@mail_account
SUPERUSER_PASSWORD=<secret>
WEBAPP_BASEURL=http://localhost:8080
```

Six Build Stages

#	Stage	Tool	Duration	Description
1	buildDB	Gradle	~30s	Creates PostgreSQL schema from merged InterMine model
2	extract_data	Python	~5 min (warm cache) / ~25 min (cold)	S3 sync + 49 FMS files + Alliance API fetchers
3	project_build	Perl (project_build)	4-7 hours	Integrates all data sources sequentially. Creates DB checkpoints at dump points.
4	postprocess	Gradle	~1 hour	create-references, do-sources, transfer-sequences, indexes, Solr search/autocomplete
5	war	Gradle	~5 min	Builds webapp WAR file
6	deploy	Gradle	~1 min	Deploys WAR to Tomcat via cargoRedeployRemote

project_build Details

The `project_build` Perl script is the core data integration engine. It processes each source in `project.xml` sequentially:

Source order (from project.xml): 1. FASTA sequences (wb, fb, zfin, mgi, rgd, xtrops, xlaevis) — **dump after mgi, xlaevis** 2. Ontologies (do, go, eco, mmo, emapa, zfa, wbbt, fbtt) — **dump after fbtt** 3. External ontologies (psi-mi, go-slim) — **dump after go-slim** 4. SGD sources (sgd, sgd-gff, sgd-gff-utr, sgd-db-utr) 5. Yeast orthologs (fungi, cgob, cglabrata, pombe, homolog-genes) 6. SGD database sources (sgd-complementation-db, sgd-complexes) — **dump after sgd-complexes** 7. SGD protein data (sgd-protein-properties, sgd-protein-ntermini) — **dump after ntermini** 8. Alliance genes, GO annotation — **dump after go-annotation** 9. Alliance disease, orthologs, alleles, expression — **dump after expression** 10. update-publications, entrez-organism, so

Checkpoints: At each dump="true" source, project_build creates a full database copy via CREATE DATABASE ... WITH TEMPLATE. This enables resume from the last checkpoint if the build fails.

Critical flags: - -b — run buildDB first (fresh build only, NOT with resume) - -l — load last checkpoint and resume - -E UTF8 — required for RDS (default SQL_ASCII breaks createdb) - Never use -T (disables server-side backups, forces slow pg_dump)

Bottleneck Sources

Source	Duration	Memory Peak	Notes
alliance-xlaevis-fasta	~30 min	~50 GB	Xenopus laevis genome, largest FASTA
fbtt	2-3 hours	~50 GB	Drosophila anatomy ontology, 180K+ terms
sgd	~25 min	~5 GB	Reads entire SGD database
update-publications	~20 min	~3 GB	Fetches from NCBI Entrez, can fail with Premature EOF

Memory Configuration

The alliancemine repo hardcodes org.gradle.jvmargs=-Xmx48g in gradle.properties. The Dockerfile patches this. The docker-compose sets: - JVM heap: -Xmx48g -Xms24g - Container limit: 56 GB (must exceed heap for native memory) - Container reservation: 24 GB

5. Data Extraction

FMS Snapshot API

GET <https://fms.alliancegenome.org/api/releaseversion/current> -> release version

GET <https://fms.alliancegenome.org/api/snapshot/release/{version}> -> snapshot with S3 URLs

The extract_data.py script runs three passes in order: 1. S3 sync of SGD/external data into /root/data/intermine/ 2. FMS snapshot download — fetches the snapshot, maps dataType/dataSubType to local filenames (FILE_MAP), downloads and decompresses gzipped files into

/root/data/fms/ and /root/data/genes/ 3. Alliance API fetch — invokes /root/alliancemine-bio-sources/scripts/fetch_all.py --out-dir /root/data/api/ with ALLIANCE_FETCH_CACHE=/root/data/cache so the SQLite cache survives container recreation

Use --skip-fms or --skip-api to re-run only one half during incidents. FMS runs first because some fetchers reuse FMS artefacts (BGI seeds, expression files, allele/disease seeds).

S3 SGD Data

```
aws s3 cp s3://agr-db-backups/alliancemine/intermine/ /root/data/intermine/ --recursive
```

This downloads ~200 MB of SGD-specific files (ontologies, protein data, yeast orthologs, GFF-UTR, idre-solver).

6. Database Management

Naming Convention

Build Type	Main DB	Profile DB
test	alliancemine_{ver}_rc{N}	alliancemine_userprofile_test
production	alliancemine_{ver}	alliancemine_userprofile

Version is sanitized: 9.0.0 -> 9_0_0. RC number is auto-incremented by querying RDS for existing databases.

Checkpoint Databases

During project_build, full DB copies are created at dump points:

```
alliancemine_9_0_0_rc18:alliance-mgi-fasta      (checkpoint after MGI FASTA)
alliancemine_9_0_0_rc18:alliance-xlaevis-fasta  (checkpoint after Xenopus)
alliancemine_9_0_0_rc18:fbbt                   (checkpoint after FBBT ontology)
...
```

Each checkpoint is 30-60 GB. With 8 checkpoints, this can consume 200+ GB. **Monitor RDS free storage during builds** and drop older checkpoints as newer ones are created.

Storage Monitoring

```
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS --metric-name FreeStorageSpace \
  --dimensions Name=DBInstanceIdentifier,Value=intermine-postgres \
  --start-time $(date -u -d '5 minutes ago' +%Y-%m-%dT%H:%M:%S) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%S) \
  --period 300 --statistics Average \
  --query 'Datapoints[0].Average' --output text | awk '{printf "%.0f GB free\n",
  ↪ $1/1024/1024/1024}'
```

Profile Database

alliancemine_userprofile is **persistent** across releases. It contains: - User accounts - Saved queries and templates - Gene lists (bags) - API tokens

It is NOT part of the build pipeline. On first webapp start with a new main DB, InterMine runs a **bag upgrade thread** that re-maps gene IDs in user lists to the new database. This blocks queries for 10-30 minutes.

7. Solr Search Indexes

Location

Solr runs directly on the Multitenant host (not in a container), port 8983.

Core Naming

Core	Purpose
alliancemine-search	Production keyword search
alliancemine-autocomplete	Production search suggestions
alliancemine-search-{version}	Per-release search index
alliancemine-autocomplete-{version}	Per-release autocomplete

Creating Cores for a New Release

InterMine does **NOT** auto-create Solr cores. The build container does it for you as a preflight step in `build_full.py` — no separate command needed for normal use. Naming mirrors the database side:

Build type	Cores
Production (BUILD_TYPE=production)	alliancemine-search-9.0.0, alliancemine-autocomplete-9.0.0
RC build (default)	alliancemine-search-9.0.0-rc99, alliancemine-autocomplete-9.0.0-rc99

`entrypoint.sh` derives `SOLR_INDEX_URL` and `SOLR_AUTOCOMPLETE_URL` from `SOLR_HOST` + release + RC so the URLs always line up with the cores the preflight creates. SSH uses the host's default keys when run from inside AWS; from a laptop, set `SOLR_SSH_KEY=/root/.ssh/AGR-ssl3.pem` in `.env` (and the compose file mounts `~/ .ssh` read-only into the container).

Preflight failure is non-fatal — postprocess will surface a clearer “core not found” later, and you can recreate cores manually and resume:

```
# Manual creation (also useful when a build is half-broken)
python3 scripts/create_solr_cores.py \
  --release 9.0.0 --rc 99 \
  --solr-host 172.31.59.87
# Inside AWS: --ssh-key omitted, defaults work
# From a laptop: add --ssh-key ~/.ssh/AGR-ssl3.pem
```

```
# Then resume the build
docker compose run --rm -e RC_NUMBER=99 alliancemine-builder build \
  --start-from postprocess --resume
```

Skip the preflight entirely with `build_full.py --skip-solr-setup` if you want to manage cores out-of-band.

Known Issue: Hardcoded Solr URLs and Hostname

The alliancemine repo has hardcoded Solr URLs in two files: - `dbmodel/resources/keyword_search.properties` -> `index.solrurl` - `dbmodel/resources/objectstoresummary.config.properties` -> `autocomplete.solrurl`

Additionally, the production WAR's `dbmodel.jar` contains `keyword_search.properties` with the old Docker hostname `agr.stage.alliancemine.solr.server`. Since Solr now runs on the host (not in a Docker container), Tomcat containers need a `/etc/hosts` entry or `--add-host` flag to resolve this:

```
# Quick fix (lost on container restart)
docker exec alliancemine sh -c \
  'echo "172.17.0.1 agr.stage.alliancemine.solr.server" >> /etc/hosts'

# Permanent fix: recreate container with --add-host
docker run -d --name alliancemine \
  --add-host agr.stage.alliancemine.solr.server:host-gateway \
  -p 8080:8080 intermine-tomcat:latest
```

These override the properties template. A PR is pending to fix this upstream. Until merged, patch them at runtime inside the build container:

```
# Inside the container:
sed -i "s|index.solrurl = .*|index.solrurl =
↪ http://172.31.59.87:8983/solr/alliancemine-search-9.0.0|" \
  /root/alliancemine/dbmodel/resources/keyword_search.properties

sed -i "s|autocomplete.solrurl = .*|autocomplete.solrurl =
↪ http://172.31.59.87:8983/solr/alliancemine-autocomplete-9.0.0|" \
  /root/alliancemine/dbmodel/resources/objectstoresummary.config.properties
```

8. WAR Deployment

Build the WAR

The WAR is built in stage 5 (`./gradlew war`). It embeds `alliancemine.properties` into `WEB-INF/classes/`.

Deploy via cargoRedeployRemote

```
docker compose run --rm alliancemine-builder release \
  --tomcat-host 172.31.59.87 \
  --tomcat-port 8082
```

Or manually from the build container:

```
./gradlew cargoRedeployRemote --stacktrace
```

This requires a running Tomcat container with the manager webapp and manager-script role.

Post-Deploy Verification

```
# From multitenant:
docker exec alliancemine-9.0.0 sh -c \
  'curl -s http://localhost:8080/alliancemine/service/version'
# Expected: 35 (InterMine version number)
```

9. Release Cutover

Pre-Release Checklist

- ☐ Full build completed (all 6 stages)
- ☐ Solr cores populated (search: ~12M docs, autocomplete: ~53K docs)
- ☐ WAR deployed to test Tomcat container
- ☐ Service version endpoint returns correctly
- ☐ Template queries work (test a few from the UI)
- ☐ Bag upgrade thread has finished
- ☐ Public gene lists recreated (if needed)

Cutover Steps

1. Update Caddy proxy config on multitenant to route to new port
2. Or: stop old container, start new one on port 8080

Rollback

The old container and database are still intact. Revert the Caddy config or restart the old container.

10. Monitoring

During Build

```
# Follow build log
tail -f /tmp/alliancemine-9.0.0-build.log

# Check RDS storage
aws cloudwatch get-metric-statistics ...

# Check container resources
docker stats --format '{{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}' | grep builder

# Check DB activity
PGPASSWORD='...' psql -h ... -d postgres -c \
  "SELECT datname, state, count(*) FROM pg_stat_activity
  WHERE datname LIKE 'alliancemine%' GROUP BY datname, state"
```

Production

```
# Check service health
curl -s https://alliancemine.alliancegenome.org/alliancemine/service/version

# Check Solr cores
curl -s 'http://172.31.59.87:8983/solr/admin/cores?action=STATUS'

# Check RDS connections
PGPASSWORD='...' psql -h ... -d postgres -c \
  "SELECT datname, count(*) FROM pg_stat_activity GROUP BY datname ORDER BY count DESC"
```

11. Troubleshooting

See docs/BUILD_TROUBLESHOOTING.md for detailed error resolutions. Key issues:

Error	Cause	Fix
Gradle daemon disappeared	OOM — container limit too low	Container limit must be > JVM heap + 8 GB
Duplicate objects for pk TelomericRepeat	SgdConverter dedup disabled	Patched in Dockerfile, upstream PR pending
XML validation failed localhost:8090	webapp.baseurl wrong	Set WEBAPP_BASEURL in .env
Cannot set superuser account	Wrong superuser name or profile DB	Set SUPERUSER_ACCOUNT + point to production profile DB
No space left on device	RDS full from checkpoint copies	Drop old checkpoint databases
Premature EOF on update-publications	NCBI Entrez connection dropped	Resume from last checkpoint